

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

Звіт

з лабораторної роботи №3
з моделювання складних систем

Варіант 1

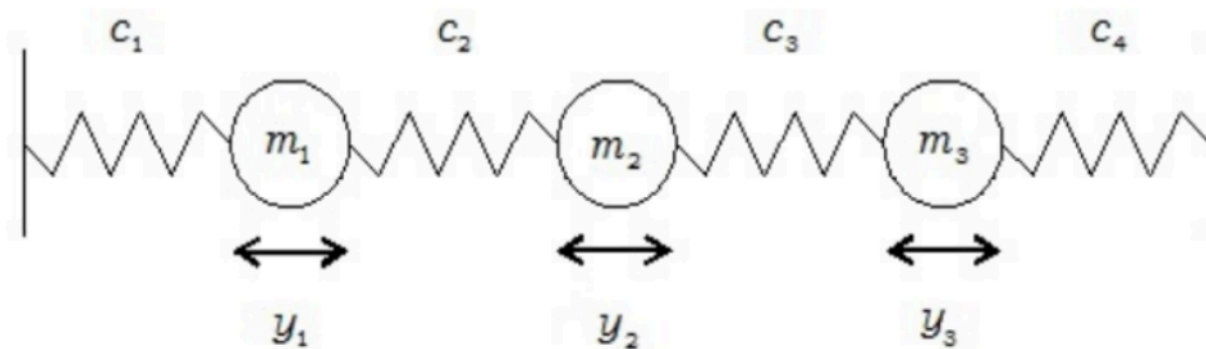
Виконала:
Студентка групи ІПС-32
Клевчук Марія Вячеславівна

Київ
2025

Параметрична ідентифікація параметрів з використанням функцій чутливості.

Постановка задачі

Для математичної моделі коливання трьох мас m_1, m_2, m_3 , які поєднані між собою пружинами з відповідними жорсткостями c_1, c_2, c_3, c_4 , і відомої функції спостереження координат моделі $\bar{y}(t), t \in [t_0, t_k]$ потрібно оцінити частину невідомих параметрів моделі з використанням функції чутливості.



Спостереження стану моделі проведені на інтервалі часу $t_0 = 0, t_k = 50, \Delta t = 0.2$.

За варіантом нам треба знайти наступні значення: $\beta = (c_1, m_1, m_2)^T$

Алгоритм

Метод параметричної ідентифікації з використанням функцій чутливості є ітераційним методом для знаходження параметрів β , які мінімізують розбіжність між розрахунковими та спостережуваними даними.

Наша мета - мінімізувати функціонал якості $I(\beta)$. Це сума квадратів різниць між спостережуваними даними $y(t)$ та розрахунковими даними $y(t, \beta)$ моделі. Чим більше значення, тим більш «неточною» є модель.

Показник якості ідентифікації параметрів β має вигляд

$$I(\beta) = \int_{t_0}^T (\bar{y}(t) - y(t, \beta))^T (\bar{y}(t) - y(t, \beta)) dt \rightarrow \min_{\beta}.$$

На початку алгоритму задаємо точність $\varepsilon > 0$ (в коді задано $10^{(-6)}$). Початкове наближення задане в умові варіанту: $\beta_0 = (0.1, 11, 23)^T$

Розв'язуємо систему, яка є математичною моделлю коливання трьох мас. На першій ітерації використовуємо значення $\beta_0, y(t_0) = \bar{y}(t_0)$.

$$\begin{aligned}\frac{d^2 y_1(t)}{dt^2} + \frac{(c_2 + c_1)}{m_1} y_1(t) - \frac{c_2}{m_1} y_2(t) &= f_1(t), \\ \frac{d^2 y_2(t)}{dt^2} - \frac{c_2}{m_2} y_1(t) + \frac{(c_2 + c_3)}{m_2} y_2(t) - \frac{c_3}{m_2} y_3(t) &= f_2(t), \\ \frac{d^2 y_3(t)}{dt^2} - \frac{c_3}{m_3} y_2(t) + \frac{(c_4 + c_3)}{m_3} y_3(t) &= f_3(t).\end{aligned}$$

Тут $f_1(t), f_2(t), f_3(t)$ – зовнішні сили.

Цю систему диференціальних рівнянь потрібно звести до нормальної форми порядку 6. Це матиме вигляд

$$\frac{d}{dt} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} \dot{y}_1 \\ \dot{y}_2 \\ \dot{y}_3 \\ \ddot{y}_1 \\ \ddot{y}_2 \\ \ddot{y}_3 \end{pmatrix} = \begin{pmatrix} x_4 \\ x_5 \\ x_6 \\ -\frac{(c_1+c_2)}{m_1}x_1 + \frac{c_2}{m_1}x_2 + f_1(t) \\ \frac{c_2}{m_2}x_1 - \frac{(c_2+c_3)}{m_2}x_2 + \frac{c_3}{m_2}x_3 + f_2(t) \\ \frac{c_3}{m_3}x_2 - \frac{(c_3+c_4)}{m_3}x_3 + f_3(t) \end{pmatrix}$$

де x - вектор стану $x = (y_1, y_2, y_3, y'_1, y'_2, y'_3)^T$

Для розв'язання цієї системи використовуємо метод Рунге-Кутти 4-го порядку.

$$\begin{aligned}\frac{dy}{dt} &= f(y, t), y(t_0) = y_0, \\ y_{n+1} &= y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),\end{aligned}$$

де

$$\begin{aligned}k_1 &= hf(y_n, t_n), \\ k_2 &= hf\left(y_n + \frac{1}{2}k_1, t_n + \frac{1}{2}h\right), \\ k_3 &= hf\left(y_n + \frac{1}{2}k_2, t_n + \frac{1}{2}h\right), \\ k_4 &= hf(y_n + k_3, t_n + h), \\ t_{n+1} &= t_n + h.\end{aligned}$$

Далі ми визначаємо, наскільки сильно кожен параметр $\beta_k = \{c_1, m_1, m_2\}$ впливає на кожну змінну стану $y_i(t)$ системи в кожний момент часу.

Це досягається шляхом розв'язання системи диференціальних рівнянь, відомої як рівняння чутливості. Воно показує **часткові похідні** вектора стану за невідомими параметрами $\beta = (c_1, m_1, m_2)$ в кожний момент часу t . Значення $U(t)$ використовується для визначення, наскільки сильно кожен із невідомих параметрів впливає на кожну змінну стану, що дозволяє обчислити напрямок і величину корекції параметрів $\Delta\beta$.

$$\frac{dU(t)}{dt} = AU(t) + \frac{\partial(Ay)}{\partial\beta}, \quad U(t_0) = 0$$

Матриця A тут - матриця Якобі, має вигляд (показано нижче) і показує, як швидкість зміни кожної змінної стану (dt/dx_i) залежить від поточного значення кожної змінної стану (x_j).

$$A = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_6} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_6}{\partial x_1} & \cdots & \frac{\partial f_6}{\partial x_6} \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ -\frac{c_1+c_2}{m_1} & \frac{c_2}{m_1} & 0 & 0 & 0 & 0 \\ \frac{c_2}{m_2} & -\frac{c_2+c_3}{m_2} & \frac{c_3}{m_2} & 0 & 0 & 0 \\ 0 & \frac{c_3}{m_3} & -\frac{c_3+c_4}{m_3} & 0 & 0 & 0 \end{pmatrix}$$

Далі обраховуємо

$$\Delta\beta_k = \left(\int_{t_0}^T U^T(t, \beta_k) U(t, \beta_k) dt \right)^{-1} \int_{t_0}^T U^T(t, \beta_k) (\bar{y}(t) - y(t, \beta_k)) dt.$$

це вектор змін, який розраховується на кожній ітерації алгоритму. Він показує, наскільки і в якому напрямку потрібно зрушити кожен із невідомих параметрів. Обчислюємо наступне наближення

$$\beta_{k+1} = \beta_k + \Delta\beta_k.$$

Алгоритм закінчуємо, коли виконані одна з умов:

1. $\Delta\beta_{k+1} < \varepsilon$;
2. $I(\beta_{k+1}) < \varepsilon$.

Хід роботи

Визначаємо всі відомі та невідомі параметри, часовий інтервал і початкове наближення.

```
# Фіксовані параметри
params_fixed = {'c2': 0.3, 'c3': 0.2, 'c4': 0.12, 'm3': 18.0}
beta_symbols = ['c1', 'm1', 'm2'] # шукаються тільки ці
beta_values = {'c1': 0.1, 'm1': 11.0, 'm2': 23.0}
```

Читаємо файл з вхідними сигналами.

```
def read_file(file_name): 1 usage
    with open(file_name, 'r') as file:
        lines = file.readlines()
        input_data = []
        for line in lines:
            values = line.strip().split()
            input_data.append([float(v) for v in values])
    return np.array(input_data).T # повертаємо у формат (N,6)
```

Ініціалізуємо матрицю A

```
def init_matr(params): 3 usages
    c1, c2, c3, c4 = params['c1'], params['c2'], params['c3'], params['c4']
    m1, m2, m3 = params['m1'], params['m2'], params['m3']
    A = np.array([
        [0, 1, 0, 0, 0, 0],
        [-(c1 + c2) / m1, 0, c2 / m1, 0, 0, 0],
        [0, 0, 0, 1, 0, 0],
        [c2 / m2, 0, -(c2 + c3) / m2, 0, c3 / m2, 0],
        [0, 0, 0, 0, 0, 1],
        [0, 0, c3 / m3, 0, -(c3 + c4) / m3, 0]
    ])
    return A
```

Розв'язуємо математичну модель в нормальній формі за допомогою алгоритму Рунге-Кутти

```
def get_y(a_matr, y_cur, h): 2 usages
    k1 = h * np.dot(a_matr, y_cur)
    k2 = h * np.dot(a_matr, y_cur + k1 / 2)
    k3 = h * np.dot(a_matr, y_cur + k2 / 2)
    k4 = h * np.dot(a_matr, y_cur + k3)
    return y_cur + (k1 + 2 * k2 + 2 * k3 + k4) / 6
```

Для розв'язку рівняння чутливості
матрицю $\frac{\partial(Ay)}{\partial\beta}$

$$\frac{dU(t)}{dt} = AU(t) + \frac{\partial(Ay)}{\partial\beta}, U(t_0) = 0 \quad \text{обчислюємо}$$

```
def finite_diff(y_vec_func, beta_keys, beta_values, delta=1e-5): 1
    n = len(y_vec_func(beta_values))
    m = len(beta_keys)
    deriv_matrix = np.zeros((n, m))

    for j in range(m):
        key = beta_keys[j]
        original_value = beta_values[key]
        beta_values[key] = original_value + delta
        y_plus = y_vec_func(beta_values)
        beta_values[key] = original_value - delta
        y_minus = y_vec_func(beta_values)
        beta_values[key] = original_value
        deriv_matrix[:, j] = (y_plus - y_minus) / (2 * delta)

    return deriv_matrix
```

Розв'язуємо рівняння чутливості, знаходячи матрицю $U(t)$

```
def get_u_matr(a_matr, b_matr, u_matr, h): 1 usage
    k1 = h * (np.dot(a_matr, u_matr) + b_matr)
    k2 = h * (np.dot(a_matr, u_matr + k1 / 2) + b_matr)
    k3 = h * (np.dot(a_matr, u_matr + k2 / 2) + b_matr)
    k4 = h * (np.dot(a_matr, u_matr + k3) + b_matr)
    return u_matr + (k1 + 2 * k2 + 2 * k3 + k4) / 6
```

Далі обчислюємо $\Delta\beta_k$ та наступний крок β_{k+1}

Перевіряємо умови завершення алгоритму та за потреби продовжуємо його.

```
def approximate(y_matr, params_fixed, beta_symbols, beta_values, eps=1e-6, h=0.2):
    ensure_logs_directory()
    start_time = time.time()
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Для збереження ітерацій
    iterations_list = []

    iteration = 0
    beta_vector = np.array([beta_values[key] for key in beta_symbols])

    def y_vec_func(b_vals_dict):
        all_params = {**params_fixed, **b_vals_dict}
        A = init_matr(all_params)
        return A @ y_approx

    while True:
        all_params = {**params_fixed, **beta_values}
        A_complete = init_matr(all_params)
        y_approx = y_matr[0]
        u_matr = np.zeros((6, len(beta_symbols)))
        integral_inv = np.zeros((len(beta_symbols), len(beta_symbols)))
        integral_mult = np.zeros(len(beta_symbols))
        quality = 0.0
```

```

for i in range(len(y_matr)):
    b_derivative = finite_diff(y_vec_func, beta_symbols, beta_values)
    integral_inv += u_matr.T @ u_matr
    integral_mult += u_matr.T @ (y_matr[i] - y_approx)
    quality += np.sum((y_matr[i] - y_approx) ** 2)
    u_matr = get_u_matr(A_complete, b_derivative, u_matr, h)
    y_approx = get_y(A_complete, y_approx, h)

integral_inv *= h
integral_mult *= h
quality *= h

delta_beta = np.linalg.inv(integral_inv) @ integral_mult
beta_vector += delta_beta
beta_values = {beta_symbols[i]: beta_vector[i] for i in range(len(beta_symbols))}

# Зберігаємо ітерацію
iter_record = {'Iteration': iteration}
for i, key in enumerate(beta_symbols):
    iter_record[key] = beta_vector[i]
iter_record['Quality'] = quality
iterations_list.append(iter_record)

iteration += 1
if quality < eps:
    break

```

Розв'язуємо систему на всьому інтервалі часу, використовуючи фінальні ідентифіковані параметри β , для порівняння з даними спостережень $y(t)$

```

def get_model_solution(params, y0, t_points, h=0.2):
    a_matrix = init_matr(params)
    y_cur = y0
    y_sol = [y0]
    for _ in range(len(t_points) - 1):
        y_cur = get_y(a_matrix, y_cur, h)
        y_sol.append(y_cur)
    return np.array(y_sol)

```

Вивід програми:

Таблиця ітерацій:

Iteration	c1	m1	m2	Quality
0	0.140320	11.926534	27.296301	3.320568e+00
1	0.139914	11.997489	27.989467	7.145839e-02
2	0.139999	11.999978	27.999920	1.212140e-05
3	0.140000	11.999991	28.000002	1.132021e-08

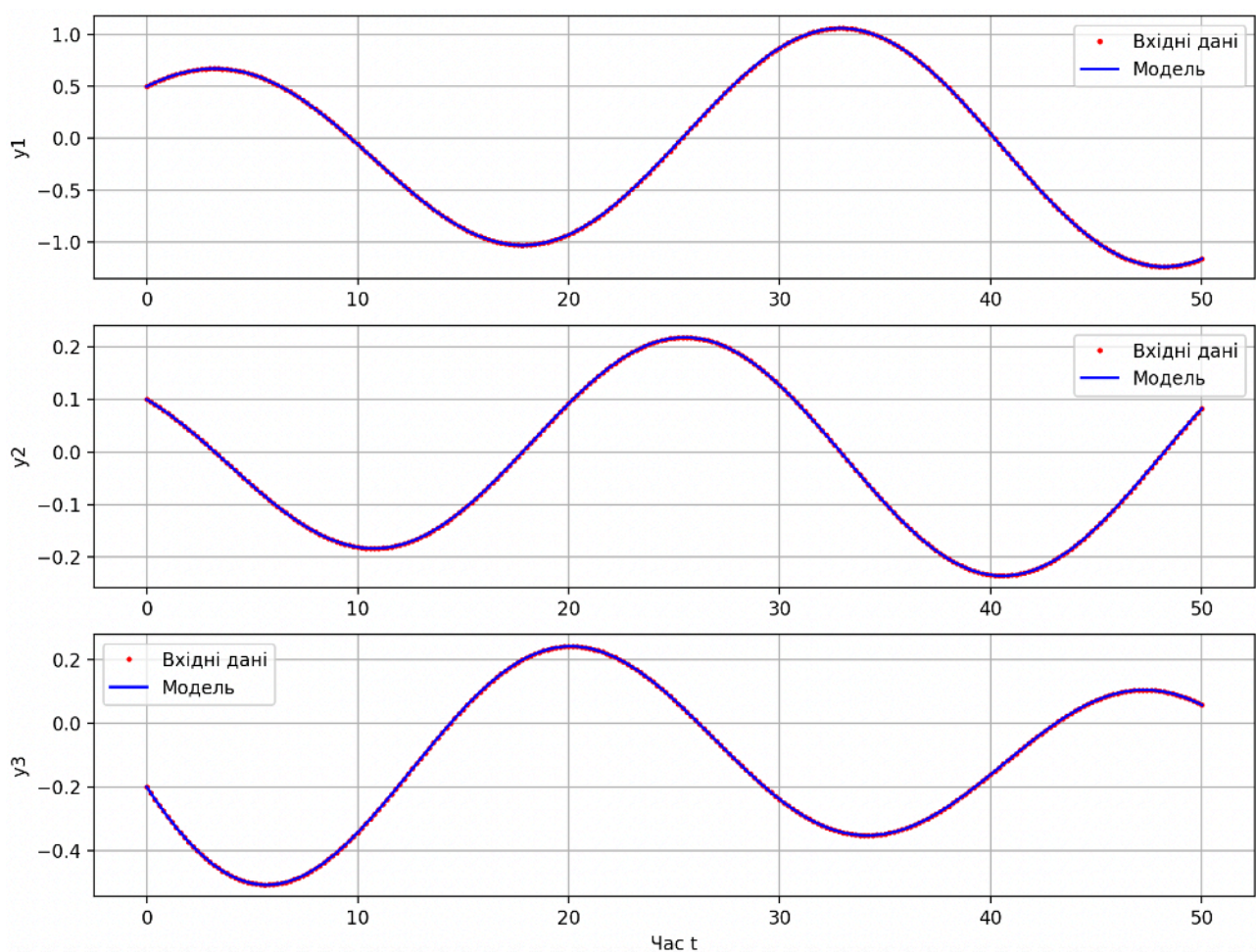
Identified parameters:

c1: 0.140000

m1: 11.999991

m2: 28.000002

Quality indicator: 1.132021e-08



З графіку бачимо, що модель вийшла дуже близькою до початкового спостереження. Отже, код реалізований правильно і обчислення коректні.

Повний код:

```
import numpy as np
import csv
from datetime import datetime
import time
import matplotlib.pyplot as plt
import os
import pandas as pd

# ----- Підготовка директорії -----
def ensure_logs_directory():
    if not os.path.exists('logs'):
        os.makedirs('logs')

# ----- Читання даних -----
def read_file(file_name):
    with open(file_name, 'r') as file:
        lines = file.readlines()
        input_data = []
        for line in lines:
            values = line.strip().split()
            input_data.append([float(v) for v in values])
        return np.array(input_data).T # повертаємо у формат (N,6)

# ----- Матриця системи -----
def init_matr(params):
```

```

    c1, c2, c3, c4 = params['c1'], params['c2'], params['c3'],
params['c4']

    m1, m2, m3 = params['m1'], params['m2'], params['m3']

    A = np.array([
        [0, 1, 0, 0, 0, 0],
        [-(c1 + c2) / m1, 0, c2 / m1, 0, 0, 0],
        [0, 0, 0, 1, 0, 0],
        [c2 / m2, 0, -(c2 + c3) / m2, 0, c3 / m2, 0],
        [0, 0, 0, 0, 0, 1],
        [0, 0, c3 / m3, 0, -(c3 + c4) / m3, 0]

    ])

    return A

# ----- RK4 для y та чутливостей
-----

def get_y(a_matr, y_cur, h):

    k1 = h * np.dot(a_matr, y_cur)

    k2 = h * np.dot(a_matr, y_cur + k1 / 2)

    k3 = h * np.dot(a_matr, y_cur + k2 / 2)

    k4 = h * np.dot(a_matr, y_cur + k3)

    return y_cur + (k1 + 2 * k2 + 2 * k3 + k4) / 6

def get_u_matr(a_matr, b_matr, u_matr, h):

    k1 = h * (np.dot(a_matr, u_matr) + b_matr)

    k2 = h * (np.dot(a_matr, u_matr + k1 / 2) + b_matr)

    k3 = h * (np.dot(a_matr, u_matr + k2 / 2) + b_matr)

    k4 = h * (np.dot(a_matr, u_matr + k3) + b_matr)

    return u_matr + (k1 + 2 * k2 + 2 * k3 + k4) / 6

```

```

# ----- Finite differences -----
def finite_diff(y_vec_func, beta_keys, beta_values, delta=1e-5):
    n = len(y_vec_func(beta_values))
    m = len(beta_keys)
    deriv_matrix = np.zeros((n, m))

    for j in range(m):
        key = beta_keys[j]
        original_value = beta_values[key]
        beta_values[key] = original_value + delta
        y_plus = y_vec_func(beta_values)
        beta_values[key] = original_value - delta
        y_minus = y_vec_func(beta_values)
        beta_values[key] = original_value
        deriv_matrix[:, j] = (y_plus - y_minus) / (2 * delta)

    return deriv_matrix

# ----- Модель рішення -----
def get_model_solution(params, y0, t_points, h=0.2):
    a_matrix = init_matr(params)
    y_cur = y0
    y_sol = [y0]
    for _ in range(len(t_points) - 1):
        y_cur = get_y(a_matrix, y_cur, h)
        y_sol.append(y_cur)

```

```

    return np.array(y_sol)

# ----- Основний алгоритм апроксимації
# -----

def approximate(y_matr, params_fixed, beta_symbols, beta_values,
eps=1e-6, h=0.2):

    ensure_logs_directory()

    start_time = time.time()

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Для збереження ітерацій

    iterations_list = []

    iteration = 0

    beta_vector = np.array([beta_values[key] for key in
beta_symbols])

    def y_vec_func(b_vals_dict):

        all_params = {**params_fixed, **b_vals_dict}

        A = init_matr(all_params)

        return A @ y_approx

    while True:

        all_params = {**params_fixed, **beta_values}

        A_complete = init_matr(all_params)

        y_approx = y_matr[0]

        u_matr = np.zeros((6, len(beta_symbols)))

        integral_inv = np.zeros((len(beta_symbols),
len(beta_symbols)))

        integral_mult = np.zeros(len(beta_symbols))

```

```

quality = 0.0

for i in range(len(y_matr)):
    b_derivative = finite_diff(y_vec_func, beta_symbols,
beta_values)

    integral_inv += u_matr.T @ u_matr
    integral_mult += u_matr.T @ (y_matr[i] - y_approx)
    quality += np.sum((y_matr[i] - y_approx) ** 2)
    u_matr = get_u_matr(A_complete, b_derivative, u_matr,
h)

    y_approx = get_y(A_complete, y_approx, h)

integral_inv *= h
integral_mult *= h
quality *= h

delta_beta = np.linalg.inv(integral_inv) @ integral_mult
beta_vector += delta_beta

beta_values = {beta_symbols[i]: beta_vector[i] for i in
range(len(beta_symbols))}

# Зберігаємо ітерацію
iter_record = {'Iteration': iteration}
for i, key in enumerate(beta_symbols):
    iter_record[key] = beta_vector[i]
iter_record['Quality'] = quality
iterations_list.append(iter_record)

iteration += 1

if quality < eps:

```

```
break
```

```
execution_time = time.time() - start_time
```

```
# Таблиця ітерацій
```

```
results_df = pd.DataFrame(iterations_list)
```

```
print("\nТаблиця ітерацій:")
```

```
print(results_df.to_string(index=False))
```

```
return beta_values, iteration, execution_time, quality,  
results_df
```

```
def plot_three_signals(t_points, measured_data, model_data,  
save_path=None):
```

```
    """
```

```
    Побудова трьох графіків на одному полотні для сигналів y1, y2,  
y3.
```

```
    Args:
```

```
        t_points (array): масив часу
```

```
        measured_data (array): вхідні дані (N,6) або (N,3) –  
беремо перші 3 сигнали
```

```
        model_data (array): модельні сигнали (N,6) або (N,3)
```

```
        save_path (str, optional): якщо вказано – зберігає графік  
у файл
```

```
    """
```

```
    labels = ['y1', 'y2', 'y3']
```

```
    plt.figure(figsize=(10, 8))
```

```

    for i in range(3):

        plt.subplot(3, 1, i + 1)

        plt.plot(t_points, measured_data[:, i], 'r.',
label='Вхідні дані', markersize=4)

        plt.plot(t_points, model_data[:, i], 'b-', label='Модель',
linewidth=1.5)

        plt.ylabel(labels[i])

        plt.legend()

        plt.grid(True)

plt.xlabel('Час t')

plt.tight_layout()

if save_path:

    plt.savefig(save_path)

    plt.close()

else:

    plt.show()

# ----- Головна функція -----
def main():

    ensure_logs_directory()

    # Данні
    y_data = read_file('./y1.txt') # (N,6)
    t_points = np.arange(len(y_data)) * 0.2

    # Фіксовані параметри

```

```

    params_fixed = {'c2': 0.3, 'c3': 0.2, 'c4': 0.12, 'm3': 18.0}
# c3 фіксоване

    beta_symbols = ['c1', 'm1', 'm2'] # шукаються тільки ці
    beta_values = {'c1': 0.1, 'm1': 11.0, 'm2': 23.0}

    result, iters, exec_time, quality, results_df =
approximate(y_data, params_fixed, beta_symbols, beta_values, 1e-6)

    final_params = {**params_fixed, **result}

    model_solution = get_model_solution(final_params, y_data[0],
t_points)

    print("\nIdentified parameters:")

    for k, v in result.items(): # ВИВОДИМО ТІЛЬКИ c1, m1, m2
        print(f"{k}: {v:.6f}")

    print(f"Quality indicator: {quality:.6e}")

    # measured_data і model_data – це масиви розміром (N,6),
беремо перші 3 сигнали

    plot_three_signals(t_points, y_data, model_solution)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

if __name__ == "__main__":
    main()

```